



Least Authority
PRIVACY MATTERS

Ledger

Security Audit Report

Midnight Network

Final Audit Report: 19 December 2025

Table of Contents

[Overview](#)

[Background](#)

[Project Dates](#)

[Review Team](#)

[Coverage](#)

[Target Code and Revision](#)

[Supporting Documentation](#)

[Areas of Concern](#)

[Findings](#)

[General Comments](#)

[Code Quality](#)

[Documentation and Code Comments](#)

[Scope](#)

[Specific Issues & Suggestions](#)

[Issue A: Non-Constant-Time Equality in HashOutput via Derived Equality Allows Timing Leakage](#)

[Issue B: VM Deviation From Specification](#)

[Issue C: VM Stack Height Limit Is Not Enforced](#)

[Issue D: effects_check Allows Extra Claimed Unshielded Spends](#)

[Issue E: Secret Material in Zswap Witnesses Is Deep-Copied and Not Zeroized](#)

[Issue F: ElGamal Ciphertexts Accept the Group Identity as the Challenge, Breaking Ciphertext-Robustness / Receiver-Binding](#)

[Issue G: Sliding-Window Reverse Index Over a Shrinking Slice Produces Duplicated Elements](#)

[Issue H: Secret Key in Coin Structure Not Zeroized](#)

[Issue I: ref_count Arithmetic Can Panic and Overflow Due to u32 → i32 Cast](#)

[Issue J: Branch Invariant Counts Empty Children but Labels Them as Nonempty](#)

[Issue K: Extension Length Invariant Violation Can Lead to State Corruption](#)

[Issue L: State Logic Allows for Invalid Merkle Tree Depth of Zero](#)

[Issue M: Incorrect Binary Length in TokenType Implementation](#)

[Issue N: Non-Canonical Zero Due to Strict Comparator in Modular Reduction](#)

[Issue O: Non-Advancing Cursor in repr_traverse May Misalign Traversal](#)

[Issue P: Usage of Vulnerable Dependencies](#)

[Suggestions](#)

[Suggestion 1: Clarify and Codify Garbage Collection Budget Policy for qc_limit](#)

[Suggestion 2: Improve Testing](#)

[Suggestion 3: Correct Error Messages](#)

[Suggestion 4: Improve Domain Separation in Coin Structure](#)

[Suggestion 5: Improve Domain Separation at Selected transient_hash Call Sites](#)

[About Least Authority](#)

[Our Methodology](#)

Overview

Background

Shielded has requested Least Authority perform multiple audits of select components of the Midnight Network: the Ledger, the Node, the Compiler, Midnight.js, the Wallet API, and the Indexer. Midnight is a fourth-generation blockchain harnessing the power of zero-knowledge cryptography. This initial audit focuses on the Ledger.

Project Dates

- **September 22, 2025 - October 17, 2025:** Initial Code Review (*Completed*)
- **October 21, 2025:** Delivery of Initial Audit Report (*Completed*)
- **TBD:** Verification Review (*Completed*)
- **TBD:** Delivery of Final Audit Report (*Completed*)

Review Team

- George Gkitsas, Security / Cryptography Researcher and Engineer
- Nikos Iliakis, Security Researcher and Engineer
- Anna Kaplan, Cryptography Researcher and Engineer
- Mirco Richter, Cryptography Researcher and Engineer
- Will Sklenars, Security Researcher and Engineer
- Burak Atasoy, Project Manager
- Jessy Bissal, Technical Editor

Coverage

Target Code and Revision

For this audit, we performed research, investigation, and review of Midnight Network's Ledger component followed by issue reporting, along with mitigation and remediation instructions as outlined in this report.

The following code repository is considered in scope for the review:

- Midnight Ledger:
<https://github.com/midnightntwrk/midnight-ledger>
 - Including the ZSwap Protocol
 - Including the DUST Protocol

Specifically, we examined the Git revision for our initial review:

- 854e7e7ccb3723d0a0584e876942b9a16ca53311

For the verification, we examined the Git revision:

- 0fff8058ff6bb75ecb3299e561279cb63d6c8516

For the review, this repository was cloned for use during the audit and for reference in this report:

- Midnight Ledger:
<https://github.com/LeastAuthority/midnight-ledger>

All file references in this document use Unix-style paths relative to the project's root directory.

In addition, any dependency and third-party code, unless specifically mentioned as in scope, were considered out of scope for this review.

Supporting Documentation

The following documentation was available to the review team:

- Midnight Architecture, code walkthrough, and Q&A:
https://drive.google.com/file/d/1Llo2-oWqOMkGJdIIATM7AMpjDY8Ab-/view?usp=drive_link
- Additional architecture repo [out of scope - provided as complementary context for the audit]:
<https://github.com/LeastAuthority/midnight-architecture>
- Websites:
 - <https://midnight.network>
 - <https://shielded.io>
- AuditList.ods (shared with Least Authority via email on 4 August 2025)

Areas of Concern

Our investigation focused on the following areas:

- Correctness of the implementation;
- Vulnerabilities within each component and whether the interaction between the components is secure;
- Whether requests are passed correctly to the network core;
- Key management, including secure private key storage and management of encryption and signing keys;
- Denial of Service (DoS) and other security exploits that would impact the intended use or disrupt the execution;
- Protection against malicious attacks and other ways to exploit;
- Inappropriate permissions and excess authority;
- Data privacy, data leaking, and information integrity; and
- Anything else as identified during the initial analysis phase.

Findings

General Comments

Midnight Ledger serves as the privacy-preserving ledger core of the Midnight platform. It is implemented in Rust and defines the transaction format, state transition rules, and proof verification pipeline, which are exposed to both native and WASM environments. The ledger implements the project's two primary assets: Zswap, which supports shielded assets and atomic swaps, and Dust, which manages transaction processing and fees. It also provides the core functionality for contract execution, transaction processing, and verification.

In our review, we audited the Zswap implementation against the [specification](#), with an emphasis on security best practices such as secret handling, and identified that secret keys are deep-copied into witnesses for zero-knowledge proofs, which are then copied and used in multiple locations ([Issue E](#)).

In addition, we audited Midnight's Zero-Knowledge Intermediate Representation (ZKIR) and identified a misbehaving reverse indexing on a shrinking slice ([Issue G](#)).

In our analysis of the cryptographic components, we reviewed the `base_crypto` and `transient_crypto` functionalities within Midnight Ledger. The `base_crypto` functionality covered, non-exhaustively, signature handling, random number generation, time capture, and encoding and conversion handling. We also reviewed the Merkle tree generation within `transient_crypto`. We identified an incorrect operator leading to a noncanonical zero representation ([Issue N](#)) and that ElGamal ciphertexts accept the group identity as the challenge, breaking ciphertext robustness and receiver binding ([Issue F](#)). In addition, the function `repr_traverse` may misalign traversal, leading to unexpected behavior ([Issue O](#)).

Furthermore, we audited `coin-structure`, `onchain-state`, `onchain-runtime` for logic consistency, where we identified an incorrect length calculation ([Issue M](#)) and a corner case in which Merkle trees with invalid lengths are accepted ([Issue L](#)). Additionally, we found that secret key material within `coin-structure` is not zeroized, and we recommend adding this additional level of protection to minimize exposure ([Issue H](#)). Moreover, we identified an opportunity for improving the application of domain separation ([Suggestion 4](#), [Suggestion 5](#)). We also reviewed the serialization logic of these components for serialization confusion and resource exhaustion but did not identify any issues.

Regarding `onchain-vm`, we audited it against the published specification, focusing on specification conformance, gas accounting, and denial-of-service hardening, and identified several divergences. We found that size-dependent gas calculations use an incorrect coefficient, runtime type tags for `Array(n)` and `BoundedMerkleTree(n)` do not match the specification formulas, and the `eq` precondition is stricter than specified by requiring both operands to be ≤ 64 bytes ([Issue B](#)). Additionally, the declared `MAX_STACK_HEIGHT` is never enforced, allowing unbounded stack growth and enabling resource exhaustion ([Issue C](#)).

Additionally, we reviewed the ledger for logic errors and specification compliance and identified that the `check_effects` function does not verify that claimed unshielded spends are not a subset of the real ones ([Issue D](#)).

We also examined the storage module, which implements a directed acyclic graph. The storage module can be configured to use one of three available backend storage engines: `parity-db`, `sqlite`, or an `in-memory` storage engine. We found the storage module to be well written overall but identified a few minor issues that could potentially lead to state corruption ([Issue K](#)) or denial of service ([Issue I](#)).

Dependencies

We examined the dependencies in the codebase and identified one vulnerable instance and one unmaintained instance. We recommend improving dependency management ([Issue P](#)).

Code Quality

We performed a manual review of the repositories in scope and found the code to be well organized and aligned with development best practices. The storage module, in particular, demonstrated high code quality, with clear organization and thorough commenting.

Tests

The project includes test suites within `coin-structure`, `onchain-state`, and `onchain-runtime`, where we identified a missing case in one of the tests ([Suggestion 2](#)). However, we were unable to generate a coverage report. We recommend incorporating test coverage analysis to identify any remaining missing cases and further strengthen the testing framework.

Documentation and Code Comments

The project includes extensive and high-quality documentation, particularly for the Zswap component, which is supported by detailed specifications and formal references, including the [Zswap paper](#), the [Kachina contract model paper](#), and additional design proposals such as the [ZKIR redesign](#). Further specifications are available in the [Midnight Ledger specification repository](#).

Additionally, while the codebase includes some code comments, the number of comments could be increased to assist with code readability and comprehension.

Scope

The core primitives of the underlying proof system were not included in this audit, as they were previously audited by an independent team, whose findings are expected to be published in the future.

We audited the Zswap codebase under the assumption that the whitelist mechanism is not used in the current implementation. Accordingly, we treated the whitelist as None, which mitigates potential synchronization issues that could otherwise affect nullifier insertion and double-spend detection.

Specific Issues & Suggestions

We list the issues and suggestions found during the review, in the order we reported them. In most cases, remediation of an issue is preferable, but mitigation is suggested as another option for cases where a trade-off could be required.

ISSUE / SUGGESTION	SEVERITY	STATUS
Issue A: Non-Constant-Time Equality in HashOutput via Derived Equality Allows Timing Leakage	Medium	Resolved
Issue B: VM Deviation From Specification	Medium	Resolved
Issue C: VM Stack Height Limit Is Not Enforced	Medium	Resolved
Issue D: effects_check Allows Extra Claimed Unshielded Spends	Medium	Resolved
Issue E: Secret Material in Zswap Witnesses Is Deep-Copied and Not Zeroized	Medium	Partially Resolved
Issue F: ElGamal Ciphertexts Accept the Group Identity as the Challenge, Breaking Ciphertext-Robustness / Receiver-Binding	Medium	Resolved
Issue G: Sliding-Window Reverse Index Over a Shrinking Slice Produces Duplicated Elements	Medium	Resolved
Issue H: Secret Key in Coin Structure Not Zeroized	Medium	Resolved
Issue I: ref_count Arithmetic Can Panic and Overflow Due to u32 → i32 Cast	Low	Resolved
Issue J: Branch Invariant Counts Empty Children but Labels Them as Nonempty	Low	Resolved
Issue K: Extension Length Invariant Violation Can Lead to State	Low	Resolved

Corruption		
Issue L: State Logic Allows for Invalid Merkle Tree Depth of Zero	Low	Resolved
Issue M: Incorrect Binary Length in TokenType Implementation	Low	Resolved
Issue N: Non-Canonical Zero Due to Strict Comparator in Modular Reduction	Low	Resolved
Issue O: Non-Advancing Cursor in repr_traverse May Misalign Traversal	Low	Resolved
Issue P: Usage of Vulnerable Dependencies	Low	Resolved
Suggestion 1: Clarify and Codify Garbage Collection Budget Policy for gc_limit	Informational	Resolved
Suggestion 2: Improve Testing	Informational	Resolved
Suggestion 3: Correct Error Messages	Informational	Resolved
Suggestion 4: Improve Domain Separation in Coin Structure	Informational	Resolved
Suggestion 5: Improve Domain Separation at transient_hash Call Sites	Informational	Partially Resolved

Issue A: Non-Constant-Time Equality in HashOutput via Derived Equality Allows Timing Leakage

Location

[base-crypto/src/hash.rs](#)

Synopsis

HashOutput is a public [u8; 32] that derives PartialEq, Eq. The derived equality performs early-exit comparison with data-dependent timing. If used for secrets, comparisons might leak the first mismatch position and could enable recovery via timing analysis.

Impact

High.

Successful exploitation might disclose secret values compared against HashOutput through timing differences, leading to unauthorized access when used for authentication or verification. If used only for public digests, impact is reduced but future misuse remains plausible.

Feasibility

Low.

An attacker must trigger comparisons of chosen inputs against a secret and measure response timing with moderate accuracy. Local or low-latency settings increase the likelihood of success. Remote attacks require repetition and averaging.

Severity

Medium.

Preconditions

A secret HashOutput must be compared to attacker-controlled data across a timing-observable boundary with repeatable queries.

Technical Details

The derived array equality performs a memcmp-like comparison and returns on the first unequal byte, thereby leaking the common-prefix length. Timing increases with each matching byte.

An attacker can adapt queries to maximize prefix matches and, by correlating latency, infer successive bytes of a 32-byte secret.

Remediation

We recommend the following steps to remediate this issue:

1. Replace the derived equality with a constant-time comparison.
2. Implement PartialEq for HashOutput using `subtle::ConstantTimeEq::ct_eq` or an XOR-accumulation approach without data-dependent branches.
3. Document the side-channel properties.
4. Audit call sites that compare secrets.

Status

The Midnight team has [resolved](#) the issue as recommended by replacing comparisons for HashOutput with a constant-time comparison through Subtle and removing all secret key equality derivations.

Verification

Resolved.

Issue B: VM Deviation From Specification

Location

[onchain-vm/src/vm.rs#L420](#)

[onchain-vm/src/vm.rs#L421](#)

[onchain-vm/src/vm.rs#L170](#)

[onchain-vm/src/vm.rs#L465](#)

[onchain-vm/src/vm.rs#L526](#)

Synopsis

The VM deviates from the published specification in several areas: The gas cost uses an incorrect coefficient for size-dependent charges, the runtime type tags for `Array(n)` and `BoundedMerkleTree(n)` do not match the specification's formulas, and `eq` rejects inputs that the specification allows by requiring both operands to be ≤ 64 bytes.

Impact

Medium.

Correct input would be rejected and invalid inputs would pass.

Severity

Medium.

Preconditions

The user must be able to craft a program triggering the affected opcodes.

Technical Details

Tag values of `Array` and `BoundedMerkleTree` are crafted with incorrect values. The values ``3 + n * 8`` and ``4 + n * 8`` (respectively) are used, while ``3 + n*32`` and ``4 + n*32`` should be used.

`Eq` opcodes check their operands for length. According to the specification, only one operand should have a length ≤ 64 bytes, but the code requires both operands to meet this condition.

The specification states that arrays must have a maximum length of 15, but currently up to 16 slots are accepted.

In the cost calculation of the `Null` opcode, the cost model of `Array` is used.

Remediation

We recommend updating the code to comply with the specification.

Status

The Midnight team has resolved all the discrepancies noted above.

Verification

Resolved.

Issue C: VM Stack Height Limit Is Not Enforced

Location

[onchain-vm/src/vm.rs#L108](#)

Synopsis

The VM defines a maximum stack depth but never enforces it. Programs can grow the stack arbitrarily.

Impact

High.

Unbounded stack growth enables a resource exhaustion denial of service (DoS).

Feasibility

Any program can emit repeated stack-growing instructions without corresponding pops.

Severity

Medium.

Preconditions

The attacker must be able to execute code that repeatedly grows the VM stack.

Technical Details

The MAX_STACK_HEIGHT constant is declared but never checked.

Remediation

We recommend bounding the stack height to MAX_STACK_HEIGHT.

Status

The Midnight team has resolved the issue by enforcing the stack limit so that an error is raised on overflow.

Verification

Resolved.

Issue D: effects_check Allows Extra Claimed Unshielded Spends

Location

[ledger/src/verify.rs#L1653](#)

Synopsis

The function enforces only one direction of the bidirectional mapping required by the specification. It verifies that every *real* unshielded spend is *claimed* but does not verify that every *claimed* unshielded spend actually exists in the transaction.

Impact

Medium.

Malicious transcripts can inflate or fabricate claimed unshielded spends.

Feasibility

High.

An attacker can craft a transcript that adds extra claimed unshielded spends.

Severity

Medium.

Preconditions

No preconditions are necessary.

Technical Details

The code checks that:

```
real_unshielded_spends.has_subset(&claimed_unshielded_spends)
```

However, the specification states that the relationship should be bidirectional, so the reverse check should also occur.

Remediation

We recommend adding the reverse check as well.

Status

After consulting with the Midnight team, our team concluded that this does not pose a threat but instead reflected an inconsistency in the specification, which has now been updated.

Verification

Resolved.

Issue E: Secret Material in Zswap Witnesses Is Deep-Copied and Not Zeroized

Location

[zswap/src/keys.rs](#)

[zswap/src/construct.rs](#)

[transient-crypto/src/proofs.rs](#)

[zswap/src/structure.rs](#)

[ledger/src/structure.rs](#)

Synopsis

`zswap/src/keys.rs` derives coin and encryption keys from a secret seed and constructs intermediate byte arrays without zeroization. The same module exposes `SecretKeys` but does not zeroize intermediates produced during derivation. `transient-crypto/src/proofs.rs` defines `ProofPreimage` with owned `Vec<Fr>` fields and no zeroization and `zswap/src/construct.rs` converts secret keys into field elements and embeds them in `ProofPreimage`, which is then duplicated across multiple data structures (`AuthorizedClaim`, `Input`, `Output`, `Transient`, and enclosing `Intent`/transaction builders). `zswap/src/structure.rs` and `ledger/src/structure.rs` store those preimages by value and clone higher-level containers before stripping proofs. These witnesses are later removed from public structures, but neither the witnesses nor their intermediate derivation buffers are scrubbed. On a compromised client, the same secret appears multiple times in heap memory and can persist after use.

Impact

Medium.

An attacker with local read access to process memory or swap can recover seed-derived material and/or the secret key's field representation from residual heap allocations, enabling reconstruction of shielded keys and subsequent unauthorized spends.

Feasibility

Medium.

The attack presumes local compromise or privileged access to process memory or crash/swap artifacts. Given the number of allocations and clones, the chance of recoverable residues increases materially.

Severity
Medium.

Preconditions

The attacker must be able to read the client's memory or disk artifacts (for example, live RAM, crash dumps, or swap). Additionally, the user must perform proving locally or via the local proof server/WASM bridge so that witnesses are present in process memory.

Technical Details

In `zswap/src/keys.rs`, `derive_encryption_secret_key` samples 64 bytes into a heap `Vec<u8>` and then clones into a `[u8; 64]` stack array. Neither buffer is explicitly zeroized on return. The `Seed` itself does not zeroize on drop, so seed bytes and intermediate keying material can linger until overwritten by unrelated allocations.

In `zswap/src/construct.rs`, the coin secret key is converted to its field representation and appended into `inputs: Vec<Fr>` when constructing a `ProofPreimage`. The preimage is then stored by value inside the action objects returned from constructors.

`transient-crypto/src/proofs.rs` defines `ProofPreimage` with owned vectors (`inputs`, `private_transcript`, `public_transcript_*`) and no zeroization or custom `Drop`, so dropping or replacing a witness does not wipe its memory.

`zswap/src/structure.rs` stores the witness by value in `AuthorizedClaim`, `Input`, `Output`, and `Transient`; `erase_proof` replaces the witness with `()`, but does not zeroize the dropped data. At the ledger layer, `Intent::erase_proofs` produces a new intent by cloning actions before erasing, transiently doubling the number of witness copies in memory.

Remediation

We recommend the following measures to mitigate this issue:

1. Treat witness material as secret and store it in zeroizing containers.
2. Wrap `ProofPreimage`'s `Vec<Fr>` fields in `zeroize::Zeroizing` (or an equivalent secrecy wrapper) and implement a `Drop` that scrubs all scalar fields.
3. Replace by-value storage of witnesses in `AuthorizedClaim`, `Input`, `Output`, and `Transient` with pointer semantics (`Box/Arc`), and remove `Clone` on types that own witnesses to move a single allocation instead of duplicating it.
4. Modify `erase_proof` to drop that single allocation.
5. Redesign `Intent::erase_proofs` to mutate in place instead of clone-then-erase, preventing transient duplicates.
6. Add optional page-locking for witness buffers on supported platforms to protect against swapping.

Status

The Midnight team has added zeroization on drop for `ProofPreimage` and related fields and moved witness storage to `Arc`-based containers to reduce deep-copied witness allocations across `Zswap` and ledger structures. Cloning and clone-then-erase patterns remain due to deduplicated containers and immutable access in the intent map, so transient duplicates can still exist. In addition, there is no page locking, and `derive_encryption_secret_key` still allocates and clones `raw_bytes` and does not scrub either buffer (`zswap/src/keys.rs#L68`).

Verification

Partially Resolved.

Issue F: ElGamal Ciphertexts Accept the Group Identity as the Challenge, Breaking Ciphertext-Robustness / Receiver-Binding

Location

[transient-crypto/src/encryption.rs#L172](#)

Synopsis

`SecretKey::decrypt` does not reject the group identity (point at infinity) as the ephemeral component `Ciphertext.c`. When `c` is the identity, the derived key material collapses to a public constant, so a sender can craft ciphertexts that decrypt successfully for every recipient, violating receiver binding.

Impact

Low.

This does not expose honest encryptions. However, it allows an attacker to produce ciphertexts that appear recipient-specific but are, in fact, publicly decryptable, which can mislead protocol logic if receiver-bound ciphertexts are assumed.

Feasibility

High.

Constructing `c = identity` and deriving the same key material from the identity's coordinates is straightforward under the current parsing/derivation behavior.

Severity

Medium.

Preconditions

The decryption path must accept `c` equal to the group identity and derive key material from its coordinates. Additionally, the attacker must be able to submit crafted ciphertexts to a recipient.

Technical Details

Because the identity element maps to fixed coordinates, the KDF output $k = H(\text{coords}(\text{identity}))$ becomes a public constant. An attacker can choose any message vector m and set the ciphertext components using that constant. For example, the attacker can compute a header from $H(k, \emptyset)$ and derive masks $H(k, i)$ to form $\text{ciph}[i] = H(k, i) + m[i]$, then send `Ciphertext{ c = identity, ciph }`. Any recipient will accept and decrypt this under their key, and any third party can perform the same decryption using the public constant. As a result, the ciphertext is valid but not receiver-bound.

Remediation

We recommend the following steps to remediate this issue:

1. Add strict validation in parsing and decryption to reject cases where `c = identity`.
2. Harden robustness by binding the recipient key into the KDF and using an AEAD DEM, while continuing to reject identity elements to block any public constant key paths.

Status

The Midnight team has added checks for the identity element during deserialization and decryption to explicitly forbid this element.

Verification

Resolved.

Issue G: Sliding-Window Reverse Index Over a Shrinking Slice Produces Duplicated Elements

Location

[zkir/src/ir_vm.rs#L146](#)

Synopsis

Within the loop, the code both reversely indexes into `inputs` using `inputs[chunks - 1 - i]` and simultaneously shrinks the same slice with `*inputs = &inputs[1..]`. Because the base of the slice moves each iteration, the “reverse” index no longer refers to the original window and repeatedly selects the current last element rather than the intended element from a fixed window.

Impact

Medium.

The routine can emit an incorrectly ordered or duplicated sequence of field elements, corrupting the logical value derived from the input. If downstream operations assume a correct per-chunk ordering (for example, parsing, encoding, hashing, or constraint construction), this can silently undermine correctness.

Feasibility

High.

The behavior is triggered deterministically whenever this code path is executed with a multi-element window.

Severity

Medium.

Preconditions

The loop processes a window with `chunks > 0` and uses `*inputs = &inputs[1..]` while accessing `inputs[chunks - 1 - i]`. The issue is easily reproduced when the input length is a multiple of `FR_BYTES_STORED` (for example, when `stray == 0`), so that chunks equal the window length.

Technical Details

Pen-and-paper example: With `FR_BYTES_STORED = 4`, `length = 12`, `stray = 0`, and `chunks = 3`, start with `inputs = [f0, f1, f2]`. Iteration 0 reads `inputs[2] = f2` then shrinks to `[f1, f2]`; iteration 1 reads `inputs[1] = f2` then shrinks to `[f2]`; iteration 2 reads `inputs[0] = f2` then shrinks to `[]`. The result is `[f2, f2, f2]` instead of the intended reversed window (for example `[f2, f1, f0]`). The root cause is computing reverse indices against a moving slice rather than a stable snapshot of the window.

Remediation

We recommend the following steps to remediate this issue:

1. Read from a stable window and only advance the cursor after selection. For example, take `let window = &inputs[..chunks]`; and iterate `window.iter().rev()` to select elements, then set `*inputs = &inputs[1..]` for the next outer step.

2. Alternatively, maintain a separate base index and avoid mutating the slice used for reverse indexing.
3. Add unit tests covering the provided example and additional cases where `stray` is nonzero.

Status

The Midnight team has resolved the issue by refraining from shrinking the slice while reverse indexing.

Verification

Resolved.

Issue H: Secret Key in Coin Structure Not Zeroized

Location

[coin-structure/src/coin.rs#L153](#)

[coin-structure/src/coin.rs#L168](#)

[coin-structure/src/coin.rs#L177](#)

Synopsis

Zeroization is absent when keys are copied or transformed, leaving residual information about them.

Impact

High.

Feasibility

Low.

Severity

Medium.

Preconditions

An initial access vector must permit direct reading of process memory or must enable indirect exfiltration of memory-resident data.

Technical Details

Secret keys require minimum exposure throughout their lifecycle. Zeroization involves clearing residual memory data associated with the keys at the end of their lifecycle. The `SecretKey` object in `Coin Structure` is not wrapped in a zeroization container. Even assuming a secure deployment, the defense-in-depth best practices dictate multiple controls for each security asset. We recommend adding this additional layer of protection for these highly critical assets.

Remediation

We recommend utilizing a zeroization library for `SecretKey`.

Status

The Midnight team has [resolved](#) the issue as recommended by correctly deriving `ZeroizeOnDrop` and removing the `Copy` derivation from the `SecretKey` struct, thereby causing all clones of this structure to be zeroized on drop.

Verification

Resolved.

Issue I: ref_count Arithmetic Can Panic and Overflow Due to u32 → i32 Cast

Location

[storage/src/backend.rs#L1084](https://github.com/leastauthority/tfa-backend-rs/blob/main/storage/src/backend.rs#L1084)

Synopsis

ref_count is stored as a u32 but is cast to i32 before applying a signed delta. If ref_count exceeds i32::MAX, the cast produces a negative value and immediately triggers the subsequent nonnegative assert, causing a process panic. Additionally, due to hash consing, individual nodes (notably the canonical empty MPT node) can accumulate very high inbound edge counts under a single root, so a u32 may be insufficient.

Impact

Medium.

An attacker or heavy workload can drive ref_count to very large values. Once ref_count surpasses i32::MAX, any operation that accesses that node's ref_count path panics, resulting in denial of service.

Feasibility

Low.

Driving refcounts extremely high requires large graphs and many updates, which renders such an attack prohibitively expensive.

Severity

Low.

Technical Details

The problematic arithmetic occurs when applying a signed delta to an unsigned counter by narrowing to i32 first:

```
let ref_count = self.ref_count as i32 + delta.ref_delta;  
assert!(ref_count >= 0, "ref count can't be negative");
```

Narrowing u32 to i32 is lossy. For any `self.ref_count > i32::MAX`, the cast yields a negative number and the assert fires.

The data model permits very high instances of ref_count under a single GC root because nodes are content-addressed and hash-consed. Identical children are deduplicated and can be referenced by many distinct parents in one trie. The canonical empty MPT node is the clearest example and has been observed with very high ref_count values in practice.

Remediation

To address this issue, we recommend the following refinements:

1. Replace the i32 cast with checked unsigned arithmetic:

```
fn apply_ref_delta(count: u64, delta: i32) -> Result<u64, StorageError>
{
    if delta >= 0 {
        count.checked_add(delta as
u64).ok_or(StorageError::RefCountOverflow)
    } else {
        count.checked_sub(delta.unsigned_abs() as
u64).ok_or(StorageError::RefCountUnderflow)
    }
}
```

2. Widen ref_count to u64 in memory and on disk.
3. Treat the canonical empty node specially to avoid a “hot counter” and large ref_count instances, either by:
 - a. making the empty node virtual: not storing it in the database, synthesizing it in memory, and skipping all refcount updates; or
 - b. keeping a single empty node object: not mutating its ref_count and not performing garbage collection on the empty node.

Status

The Midnight team has resolved the issue by using 64-bit unsigned integers instead of 32-bit and has removed the potentially lossy cast to a signed 32-bit integer.

Verification

Resolved.

Issue J: Branch Invariant Counts Empty Children but Labels Them as Nonempty

Location

[storage/src/merkle_patricia_trie.rs#L1353](https://github.com/leastauthority/midnight-storage/src/merkle_patricia_trie.rs#L1353)

Synopsis

The check_invariant for Node::Branch is intended to reject branches with fewer than two nonempty children, but the implementation mistakenly counts empty children. As a result, branches with 15 to 16 nonempty children are incorrectly rejected, and branches with fewer than two nonempty children can incorrectly pass this check.

Impact

Medium.

Accepting malformed Branch nodes (with fewer than two nonempty children) defeats a core structural invariant used throughout the trie logic (for example, shape normalization to Extension when only one child should remain). This can:

- allow malformed tries to be deserialized without error;
- lead to stale or incorrect annotations being considered valid in later operations; and

- cause unexpected control flow and harder-to-reason-about performance and pathological shapes.

While this is not a direct memory safety issue, it undermines integrity checks that protect against corrupted or crafted on-disk trees.

Feasibility

Low.

Severity

Low.

Technical Details

The variable `non_empty_children` is computed with:

```
.filter(|child| matches!(*child, Node::Empty)),
```

which counts empty children, not nonempty ones.

The subsequent check:

```
if non_empty_children < 2 { /* error */ }
```

therefore triggers an error only when there are fewer than two empty children (that is, 14 to 16 nonempty children), which is the opposite of the intended condition.

This produces both false positives (dense, valid branches rejected) and false negatives (underpopulated branches accepted).

Remediation

We recommend correcting the logic to count nonempty children rather than empty.

Status

The Midnight team has resolved the issue by updating the invariant to count nonempty children rather than empty ones.

Verification

Resolved.

Issue K: Extension Length Invariant Violation Can Lead to State Corruption

Location

[storage/src/merkle_patricia_trie.rs#L619-L621](#)

[storage/src/merkle_patricia_trie.rs#L662-L666](#)

Synopsis

`Extension.compressed_path` is intended to be ≤ 255 nibbles. During prune, two code paths concatenate path segments without rechunking, so an `Extension` that already has a length of 255 can grow to 256 or more nibbles. When serialized, the length is cast to `u8` and wraps modulo 256, causing deserialization to reconstruct an incorrect (shorter) path and silently corrupt the trie's structure.

Impact

Medium.

The issue could lead to state corruption across `serialize` and `deserialize` boundaries: nodes under an affected `Extension` could be mapped to the incorrect key prefix, resulting in incorrect lookups.

Feasibility

Low.

Severity

Low.

Technical Details

An inline comment in `Extension.compressed_path` states the intention for `compressed_path.length` to be no longer than 255:

```
compressed_path: std::vec::Vec<u8>, // with a length no longer than 255
```

In the `prune` function, within the `Node::Branch` arm, collapsing a single remaining child that is an `Extension` performs the following operation:

```
compressed_path: once(path_head as u8)
    .chain(compressed_path.iter().copied())
    .collect()
```

If `compressed_path.len() == 255`, the new length becomes 256.

However, serialization writes the nibble count as a `u8`:

```
u8::serialize(&(compressed_path.len() as u8), writer)?;
```

`deserialize` then reconstructs only the number of nibbles indicated by that length byte, which after wrapping results in a truncated path.

Remediation

We recommend splitting any `Extension` path longer than 255 nibbles into a chain of `Extension` nodes, each containing ≤ 255 nibbles.

Status

The Midnight team has resolved the issue by adding an extension helper that chunks the path into 255-nibble pieces and builds a chain of `Extension` nodes.

Verification

Resolved.

Issue L: State Logic Allows for Invalid Merkle Tree Depth of Zero

Location

[onchain-state/src/state.rs#L417](https://github.com/leastauthority/ledger/blob/main/src/state.rs#L417)

Synopsis

A logic error in the handling of `MerkleTree` height permits zero-height trees, despite the specification requiring heights strictly greater than zero.

Impact

Low.

Construction and acceptance of zero-height trees is allowed, potentially leading to logic errors or application panics for components expecting strictly positive tree heights.

Feasibility

Low.

Severity

Low.

Preconditions

The attacker must be able to provide or force a Merkle tree of zero height as input.

Technical Details

The lower bound on the `MerkleTree` height is not enforced in the runtime invariant despite the [specification](#). The `StateValue::BoundedMerkleTree` variant is documented as “Merkle tree with $0 < \text{height} \leq 32$,” but `StateValue::invariant` only checks `bmt.height() > 32` and that `bmt.root().is_some()`, omitting a check that `height >= 1`.

Remediation

We recommend adding a check to verify that the Merkle trees in `StateValue::invariant` have `height >= 1`.

Status

The Midnight team has added the required checks [here](#), [here](#), and [here](#).

Verification

Resolved.

Issue M: Incorrect Binary Length in TokenType Implementation

Location

[coin-structure/src/coin.rs#L553](#)

Synopsis

A discrepancy was identified in the `BinaryHashRepr` implementation of the `TokenType` enum. The `binary_len()` method underreports the serialized length of non-Dust variants by one byte. While the binary representation writes a one-byte discriminant followed by a 32-byte hash, the length method reports only 32 bytes.

Impact

Low.

The inconsistency could lead to downstream serialization or buffer allocation errors.

Severity

Low.

Preconditions

No preconditions are necessary.

Technical Details

In both the Unshielded and Shielded cases, the `binary_repr` method emits 33 bytes: one-byte discriminant + 32-byte hash value.

However, `binary_len()` advertises only 32 bytes, creating a mismatch between the reported and actual serialized sizes.

Remediation

We recommend correcting the length method and adding a regression test.

Status

The Midnight team has [resolved](#) the issue as recommended.

Verification

Resolved.

Issue N: Non-Canonical Zero Due to Strict Comparator in Modular Reduction

Location

[transient-crypto/src/curve.rs#L418](#)

Synopsis

The comparison that checks whether a field/scalar value exceeds the modulus uses a strict operator instead of an inclusive one. As a result, an input equal to the modulus is treated as valid rather than reduced/rejected, creating a second encoding of zero and breaking canonicity.

Impact

Low.

An attacker can supply alternative encodings of zero to bypass canonical-encoding checks, potentially enabling malleability or equality/normalization inconsistencies, but not direct key or secret recovery.

Feasibility

High.

If untrusted inputs can reach this reduction/validation step, crafting a value equal to the modulus is trivial.

Severity

Low.

Preconditions

The vulnerable path must accept externally controlled field or scalar elements without a subsequent canonicalization pass, and those values must be consumed in contexts that assume unique encodings.

Technical Details

In modular arithmetic over a prime modulus, any input $x \geq p$ must be reduced or rejected. Permitting $x=p$ admits a non-canonical representative of zero ($p \equiv 0 \pmod{p}$). The comparator at the referenced line should therefore use “ $\geq$ ” rather than “ $>$ ” to enforce uniqueness of encodings and preserve invariants in downstream curve/scalar operations.

Remediation

We recommend changing the comparator.

Status

The Midnight team has resolved the issue by introducing the correct `rhs >= embedded_modulus` comparator.

Verification

Resolved.

Issue O: Non-Advancing Cursor in `repr_traverse` May Misalign Traversal

Location

[transient-crypto/src/fab.rs#L222](https://github.com/transient-crypto/src/fab.rs#L222)

Synopsis

In `repr_traverse`, the `AlignmentSegment::Atom(_)` branch advances `atom_slice` (for example, `atom_slice[1..]`), while the `AlignmentSegment::Option(_)` branch recurses on the same `atom_slice` without consuming an element.

Impact

Low.

A crafted or unexpected representation could be accepted or rejected incorrectly due to misaligned traversal, weakening validation but not directly yielding control flow or secrets.

Feasibility

Low.

The maintainer indicates this path is largely dormant and intended for future capabilities, so triggering conditions are limited in current builds.

Severity

Low.

Preconditions

The `Option` segment must be reachable at runtime (for example, configurations or inputs that exercise `AlignmentSegment::Option` within `repr_traverse`).

Technical Details

The function’s match logic consumes one atom when encountering `AlignmentSegment::Atom(_)` but, for `AlignmentSegment::Option(_)`, makes a recursive call using the unchanged `atom_slice`. Without a corresponding consumption step on successful option selection (or a guard ensuring no

repeated recursion on the same cursor), the traversal can become out of sync with the representation, leading to reprocessing or skipping mismatched atoms and producing incorrect results.

Remediation

We recommend the following steps to remediate this issue:

1. Modify the `Option` branch to advance the cursor when an alternative is selected, or restructure the recursion to consume atoms consistently across all branches.
2. Add unit tests asserting cursor progression invariants, and introduce defensive checks, such as depth or progress guards, to prevent non-progressing recursion.
3. Return an explicit error for unsupported paths to avoid ambiguous behavior.

Status

The Midnight team has updated `repr_traverse` to consume the `Option` discriminant and advance the cursor before recursion, and added tests to detect nonadvancing traversal and length mismatches.

Verification

Resolved.

Issue P: Usage of Vulnerable Dependencies

Location

[midnight-ledger/blob/Cargo.toml](https://github.com/midnight-ledger/blob/Cargo.toml)

Synopsis

Analyzing the codebase with `cargo audit` for dependency versions shows two vulnerabilities.

Impact

Using unmaintained dependencies or packages with known vulnerabilities may lead to critical security vulnerabilities in the codebase.

Severity

Low.

Technical Details

The following dependency vulnerabilities were identified:

Crate: `tracing-subscriber`
Version: `0.3.19`
Title: Logging user input may result in poisoning logs with ANSI escape sequences
Date: `2025-08-29`
ID: `RUSTSEC-2025-0055`
URL: <https://rustsec.org/advisories/RUSTSEC-2025-0055>
Solution: Upgrade to `>=0.3.20`

Crate: `paste`
Version: `1.0.15`
Title: `paste` - no longer maintained
Date: `2024-10-07`
ID: `RUSTSEC-2024-0436`

URL: <https://rustsec.org/advisories/RUSTSEC-2024-0436>

Warning: unmaintained

Remediation

We recommend upgrading the two dependencies.

Status

The Midnight team has [replaced](#) the unmaintained library paste with pasteey as a direct dependency. However, the indirect dependencies for this library remain unaddressed. Our team agrees with the Midnight team that, because this is an informative advisory for an unmaintained library, this approach is acceptable.

Furthermore, the Midnight team has added cargo audit to CI to obtain systematic visibility into dependency-related vulnerabilities.

Verification

Resolved.

Suggestions

Suggestion 1: Clarify and Codify Garbage Collection Budget Policy for gc_limit

Location

[src/delta_tracking/mod.rs#L85](#)

Synopsis

The garbage collection step budget passed to gc_rmap is derived from a RunningCost that ignores deletes (the deletes argument to cpu_cost is hard coded to zero). This implicitly causes the garbage collection progress to depend on write volume rather than delete pressure. The policy is not stated and appears inconsistent with the presence of a delete term in the cost model (gc_rmap_coeff_keys_removed_size). This can result in delete-heavy transitions cleaning up slowly.

Mitigation

We recommend establishing a clear garbage collection policy that is delete-driven, write-amortized, or hybrid and articulating this choice explicitly in the code. We also optionally recommend implementing a fixpoint loop under a bounded budget.

Status

The Midnight team has reviewed its garbage collection policy, determined it sufficient, and documented the design intent.

Verification

Resolved.

Suggestion 2: Improve Testing

Location

[onchain-state/src/state.rs#L145](#)

Synopsis

Testing does not account for all test cases. For example, the case of 40 [here](#) will never be triggered by the [corresponding test](#).

Mitigation

We recommend fixing the specific missing case and using test coverage to identify other missing cases.

Status

The Midnight team has [corrected the specific test](#) that was provided as an example of missing coverage. The team also reported plans to add code coverage reporting, either in PR checks if performance permits or through periodic tests on main. In addition, the team is evaluating how best to surface coverage information so that it is acted upon, with the current approach involving PR feedback reports showing how a change affects coverage metrics. These efforts are not yet implemented but are part of an independent QA-led initiative. We find these planned improvements positive, and the Midnight team's recognition of the importance of continuous coverage metrics leads us to consider this issue resolved.

Verification

Resolved.

Suggestion 3: Correct Error Messages

Location

[coin-structure/src/coin.rs#L395](#)

Synopsis

The error message is incorrect, resulting in error misattribution. When the size is 1 and the tag is unsupported, the error message does not indicate that the tag is unsupported.

Mitigation

We recommend producing the correct error for the corresponding triggering condition.

Status

The Midnight team has [implemented](#) the mitigation as recommended.

Verification

Resolved.

Suggestion 4: Improve Domain Separation in Coin Structure

Location

[coin-structure/src/coin.rs#L169](#)

[coin-structure/src/coin.rs#L637](#)

[coin-structure/src/coin.rs#L648](#)

Synopsis

The code builds several cryptographic identifiers (for example, coin commitments and nullifiers) by concatenating a binary encoding of structured data and then appending a short ASCII tag, such as `b"mdn:cc"` or `b"mdn:cn,"` before calling `persistent_hash`.

Length extension attacks are not possible since the data are of fixed length.

With Merkle–Damgård hashes such as SHA-256 used by `persistent_hash`, placing the label at the beginning is the standard and more secure practice. While still collision resistant in theory due to SHA-256's properties, a prefixed domain separator causes every subsequent block to be processed under a different internal state derived from that unique prefix. Prefix-based domain separation provides independent hash domains, while suffix tagging does not isolate internal states.

An additional improvement is to include a domain version so that encodings can be changed later without risking cross-domain collisions.

The data should be represented by a type that includes the data and the separator instead of a byte stream. This follows best software development practices and future-proofs the codebase. Furthermore, it is recommended that structured domain separation be used by adding the length of each field and, optionally, a tag denoting its type for all data fields.

Mitigation

We suggest the following improvements:

1. Switch to domain separation prefixing;
2. Add a domain version field; and
3. Organize the domain separation by adding a length and a type tag for each field that is hashed.

Status

The Midnight team has [updated the implementation](#) to use domain-separation prefixing and to organize the different domains with tags. For technical reasons, adding self-describing data for each field was not the preferred approach. Instead, the design establishes domain separators that uniquely define the expected input format. Our team therefore considers the issue resolved.

Verification

Resolved.

Suggestion 5: Improve Domain Separation at `transient_hash` Call Sites

Location

[transient-crypto/src/hash.rs#L74-L80](#)

[ledger/src/dust.rs#L457-L459](#)

[ledger/src/dust.rs#L544-L548](#)

[transient-crypto/src/merkle_tree.rs#L149-L156](#)

[zswap/src/lib.rs#L24-L31](#)

[onchain-runtime-wasm/src/primitives.rs#L225-L241](#)

Synopsis

`transient_hash` intentionally lacks a built-in domain separator. Several call sites hash tuples or pairs without an explicit constant prefix (for example, Merkle internal nodes, proof-binding inputs, dust generation information, Zswap ciphertext coordinates, and WASM communications commitments). Although context fields reduce immediate risk, the absence of a fixed domain tag allows potential cross-protocol or cross-context mix-and-match if identical preimages occur elsewhere or in future extensions.

Mitigation

We recommend the following measures to mitigate this behavior:

1. Prepend stable ASCII domain tags to the preimages at the listed sites (for example, `midnight:comm-commit`, `mdn:merkle-node`, `mdn:dust:gen`, `midnight:zswap:ciphertext`).
2. Version the domain tags in documentation to prevent silent collisions.

Status

The Midnight team has added explicit domain separation for the Dust spend proof binding in `ledger/src/dust.rs` and for Zswap ciphertext coordinate hashing in `zswap/src/lib.rs` by prefixing fixed ASCII tags. However, no fixed domain tag was added at the Merkle internal node hashing sites in `transient-crypto/src/merkle_tree.rs` or at the WASM communications commitment hashing site in `onchain-runtime-wasm/src/primitives.rs`. In addition, versioned domain tags are not documented.

Verification

Partially Resolved.

About Least Authority

We believe that people have a fundamental right to privacy and that the use of secure solutions enables people to more freely use the Internet and other connected technologies. We provide security consulting services to help others make their solutions more resistant to unauthorized access to data and unintended manipulation of the system. We support teams from the design phase through the production launch and after.

The Least Authority team has skills for reviewing code in multiple Languages, such as C, C++, Python, Haskell, Rust, Node.js, Solidity, Go, JavaScript, ZoKrates, and circom, for common security vulnerabilities and specific attack vectors. The team has reviewed implementations of cryptographic protocols and distributed system architecture in cryptocurrency, blockchains, payments, smart contracts, zero-knowledge protocols, and consensus protocols. Additionally, the team can utilize various tools to scan code and networks and build custom tools as necessary.

Least Authority was formed in 2011 to create and further empower freedom-compatible technologies. We moved the company to Berlin in 2016 and continue to expand our efforts. We are an international team that believes we can have a significant impact on the world by being transparent and open about the work we do.

For more information about our security consulting, please visit <https://leastauthority.com/security-consulting/>.

Our Methodology

We like to work with a transparent process and make our reviews a collaborative effort. The goals of our security audits are to improve the quality of systems we review and aim for sufficient remediation to help protect users. The following is the methodology we use in our security audit process.

Manual Code Review

In manually reviewing all of the code, we look for any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis

Our audit techniques include manual code analysis, user interface interaction, and whitebox penetration testing. We look at the project's website to get a high level understanding of what functionality the software under review provides. We then meet with the developers to gain an appreciation of their vision of the software. We install and use the relevant software, exploring the user interactions and roles. As we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation. We hypothesize what vulnerabilities may be present and possibly resulting in Issue entries, then for each, we follow the following Issue Investigation and Remediation process.

Documenting Results

We follow a conservative and transparent process for analyzing potential security vulnerabilities and seeing them through successful remediation. Whenever a potential issue is discovered, we immediately create an Issue entry for it in this document, even before having verified the feasibility and impact of the issue. This process is conservative because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyze the feasibility of an attack in a live system.

Suggested Solutions

We search for immediate and comprehensive mitigations that live deployments can take, and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinized by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our Initial Audit Report, and before we perform a verification review.

Before our report, including any details about our findings and the solutions are shared, we like to work with your team to find reasonable outcomes that can be addressed as soon as possible without an overly negative impact on pre-existing plans. Although the handling of issues must be done on a case-by-case basis, we always like to agree on a timeline for a resolution that balances the impact on the users and the needs of your project team.

Resolutions & Publishing

Once the findings are comprehensively addressed, we complete a verification review to assess that the issues and suggestions are sufficiently addressed. When this analysis is completed, we update the report and provide a Final Audit Report that can be published in whole. If there are critical unaddressed issues, we suggest the report not be published and the users and other stakeholders be alerted of the impact. We encourage that all findings be dealt with and the Final Audit Report be shared publicly for the transparency of efforts and the advancement of security learnings within the industry.